DATA ENGINEERING | INTERNAL

TECHNICAL DESIGN DOCUMENT

Open-Meteo Weather End-to-End ETL Pipeline

Design, data mapping, data quality framework, SQL and operations guide

Document ID DE-ETL-WX-001 | Version 1.0 | 08 Nov 2026

DOCUMENT CONTROL

Document title	Technical Design Document: Open-Meteo Weather ETL Pipeline
Document ID	DE-ETL-WX-001
Version / status	1.0 | Final
Date	08 Nov 2026
Author	[Your Name], Data Engineer
Classification	Internal
Project	Hourly weather ingestion for Hyderabad into SQL Server

Revision history

Version	Date	Author	Description
1.0	08 Nov 2026	[Your Name]	Initial release: design, mapping, data quality framework, SQL scripts and runbook

Approvals

Role	Name	Signature	Date
Prepared by	[Your Name]		
Reviewed by			
Approved by			

Table of Contents

1. Purpose and Scope . . . 3

2. Solution Architecture . . . 4

3. Source and Configuration . . . 5

4. Data Mapping . . . 6

5. ETL Process Design . . . 7

6. Data Quality Framework . . . 8

7. SQL Scripts . . . 9

8. Operations Guide . . . 10

9. Testing and Reconciliation . . . 11

10. Known Issues, Risks and Recommendations . . . 12

Appendix A. Glossary . . . 12

1. Purpose and Scope

1.1 Purpose

This document describes the design of an automated ETL (extract, transform, load) pipeline that collects hourly weather data for Hyderabad from the Open-Meteo API, checks its quality, stores it as files and loads it into a SQL Server table. It is the reference for anyone who needs to run, support or extend the pipeline.

1.2 Scope

In scope	Out of scope
Extraction of hourly temperature, humidity, wind speed and precipitation for one location (24 hours)	Weather forecasting or modelling
Transformation and validation in Python (Pandas)	Multi-location or historical backfill (see section 10)
Export to JSON and CSV, and load into SQL Server	Scheduling and orchestration (see section 10)
Logging, retries and a post-load reconciliation	Dashboards and reporting layer

1.3 Objectives

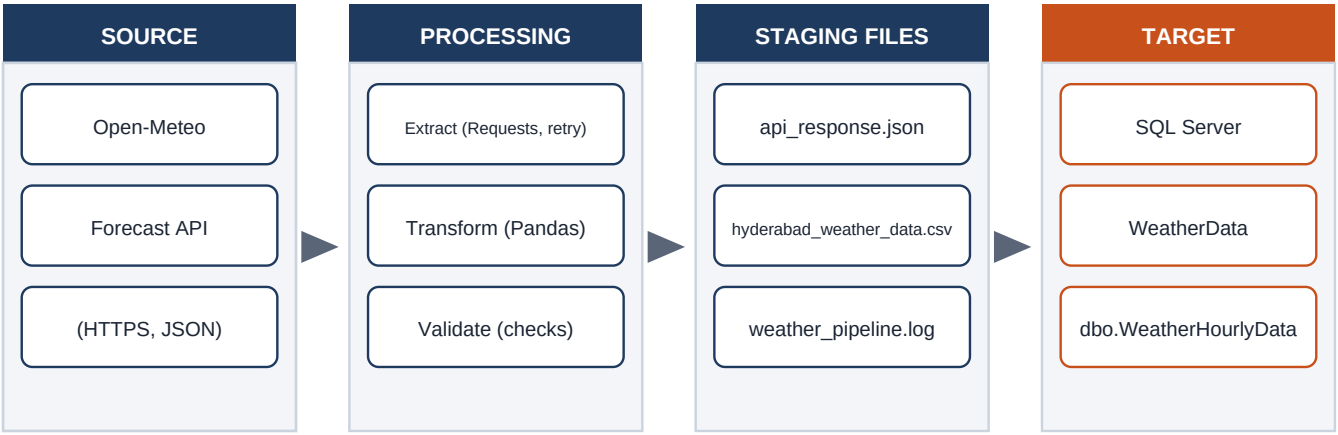
- **Reliable:** transient API failures are retried and every failure is logged with its full error.
- **Verifiable:** data is checked before loading and the loaded row count is reconciled afterwards.
- **Reusable:** raw and clean data are both kept as files, so a run can be audited or replayed.

1.4 Assumptions

- The Open-Meteo forecast endpoint is available without an API key for this usage.
- The target database WeatherData exists and the running user can create and write to dbo tables.
- One run processes one forecast day (24 hourly rows) for one location.

2. Solution Architecture

Data moves left to right: the API response is processed in Python, saved to staging files, and loaded into the target table.



CROSS-CUTTING: logging | retry policy | error handling | execution time tracking | Windows authentication

2.1 Components

Component	Technology	Responsibility
Orchestration	Jupyter Notebook	Runs the pipeline steps in order and shows the output of each
Extraction	Python Requests, urllib3 Retry	Calls the API with timeout and automatic retries
Processing	Pandas	Builds the DataFrame, converts types and profiles the data
Logging	Python logging	Writes timestamped INFO and ERROR messages to weather_pipeline.log
Staging	JSON and CSV files	Keeps the raw response and the clean dataset
Database access	PyODBC, SQLAlchemy	Connects to SQL Server and loads with to_sql
Target	SQL Server (WeatherData)	Stores the hourly records in dbo.WeatherHourlyData

2.2 Non-functional design

Area	Design decision
Reliability	Up to 3 retries on HTTP 429, 500, 502, 503 and 504 with increasing wait; 30 s timeout
Observability	File log for every stage plus records processed and execution time at the end
Security	Windows authentication to SQL Server; no passwords or keys in code
Auditability	Raw API response saved unchanged as api_response.json

3. Source and Configuration

3.1 Source system

System	Open-Meteo Forecast API
Endpoint	https://api.open-meteo.com/v1/forecast
Method / format	HTTP GET, JSON response
Location	Latitude 17.3850, longitude 78.4867 (Hyderabad). The API returns its nearest grid point: 17.3989, 78.4571, elevation 505 m
Variables	temperature_2m, relative_humidity_2m, wind_speed_10m, precipitation (hourly)
Window / timezone	forecast_days = 1; timezone Asia/Kolkata (UTC+05:30)

3.2 Resilience settings

Parameter	Value	Purpose
Retry total	3	Maximum retry attempts
backoff_factor	1	Wait grows between attempts
status_forcelist	429, 500, 502, 503, 504	Retry only on temporary errors
allowed_methods	GET	Safe to repeat
Timeout	30 seconds	Stops a hanging request

3.3 Target system

Server type	SQL Server (local SQL Express instance)
Database	WeatherData
Table	dbo.WeatherHourlyData
Driver	ODBC Driver 17 for SQL Server
Authentication	Windows authentication (Trusted_Connection)
Load method	pandas to_sql, if_exists = append, index = False, through a SQLAlchemy mssql+pyodbc engine

3.4 Output artefacts

Artefact	Description
api_response.json	Full raw API response, indented
hyderabad_weather_data.csv	Clean dataset, 24 rows by 5 columns
weather_pipeline.log	Run log, INFO level and above

4. Data Mapping

4.1 Source-to-target mapping

ID	Source field	Transformation	Target column	Target type
M-01	hourly.time	Parse ISO 8601 text to datetime (pd.to_datetime)	time	DATETIME2(0)
M-02	hourly.temperature_2m	None (direct copy)	temperature_2m	FLOAT
M-03	hourly.relative_humidity_2m	None (direct copy)	relative_humidity_2m	INT
M-04	hourly.wind_speed_10m	None (direct copy)	wind_speed_10m	FLOAT
M-05	hourly.precipitation	None (direct copy)	precipitation	FLOAT

Fields outside the hourly block (latitude, longitude, elevation, units) are kept in api_response.json and are not loaded to SQL Server.

4.2 Data dictionary

Column	Description	Unit	Pandas type	Null ?	Example
time	Hour the reading applies to, local time	datetime	datetime64[ns]	No	2026-11-08 00:00
temperature_2m	Air temperature at 2 m above ground	degrees C	float64	No	23.4
relative_humidity_2m	Relative humidity at 2 m	%	int64	No	51
wind_speed_10m	Wind speed at 10 m above ground	km/h	float64	No	7.1
precipitation	Rain, showers and snow in the previous hour	mm	float64	No	0.0

4.3 Business keys

Each row is one hour at one location, so **time** uniquely identifies a row in the current single-location design. If more locations are added, a location column must join the key (see section 10).

5. ETL Process Design

The notebook steps are grouped into process steps. Each step has a defined failure behaviour.

ID	Step	Description	On failure
E-01	Configure	Set endpoint, location, variables, timezone	n/a
E-02	Session	Create HTTP session with retry adapter	n/a
E-03	Extract	GET request, timeout 30 s, check HTTP status, parse JSON	Log full error, stop run
T-01	Build table	Convert hourly JSON to a DataFrame	Stop run
V-01	Validate	Shape, dtypes, nulls, duplicates, statistics	Log full error, stop run
T-02	Convert	Convert time column to datetime	Stop run
X-01	Export	Write JSON and CSV files	Stop run
L-01	Connect	Open PyODBC connection, confirm database name	Stop run
L-02	Load	Insert DataFrame into dbo.WeatherHourlyData	Stop run
L-03	Reconcile	COUNT(*) in SQL Server compared with rows processed	Investigate
M-01	Monitor	Log records processed, execution time and final status	n/a

5.1 Error handling

Situation	Handling
Temporary API error (HTTP 429 or 5xx)	Automatic retry, up to 3 attempts
Network failure or bad response	Exception logged with stack trace by logger.exception, then raised
Invalid JSON	ValueError caught, logged and raised
Validation error	Logged and raised before any load happens

5.2 Execution order

Configure, extract, build table, validate, convert, export, load, reconcile, summarise. Validation runs before the load, so a failed check prevents bad data from reaching the database.

6. Data Quality Framework

Rules are grouped by quality dimension. Results are from the run documented in section 9.

Rule	Dimension	Logic	Severity	Result
DQ-01	Completeness	No NULL in any of the 5 columns (df.isnull().sum())	High	0 nulls: **Pass**
DQ-02	Uniqueness	No duplicate rows (df.duplicated().sum())	High	0 duplicates: **Pass**
DQ-03	Completeness	24 hourly rows for the forecast day	High	24 rows: **Pass**
DQ-04	Structure	Five expected columns present in the expected order	High	Match: **Pass**
DQ-05	Validity	time is datetime64 after conversion; others numeric	High	Match: **Pass**
DQ-06	Validity	Humidity between 0 and 100 %	Medium	29 to 52: **Pass**
DQ-07	Validity	Temperature between -60 and 60 C	Medium	21.6 to 31.1: **Pass**
DQ-08	Validity	Wind speed and precipitation not negative	Medium	6.0 min, 0.0 min: **Pass**
DQ-09	Consistency	Rows loaded equal rows processed	High	24 = 24: **Pass**

Rules DQ-01 to DQ-05 and DQ-09 run in the notebook. Rules DQ-06 to DQ-08 are evaluated here against the observed minimum and maximum from df.describe(); the SQL in section 7 turns them into repeatable checks.

6.1 Statistical profile

Statistic	temperature_2m (C)	relative_humidity_2m (%)	wind_speed_10m (km/h)	precipitation (mm)
count	24	24	24	24
mean	26.06	40.29	8.78	0.00
std	3.37	9.03	1.95	0.00
min	21.60	29.00	6.00	0.00
25%	22.95	31.75	7.30	0.00
50%	25.40	40.00	8.35	0.00
75%	29.08	51.00	9.80	0.00
max	31.10	52.00	13.60	0.00

Observation: precipitation is 0.0 mm for all 24 hours. This is plausible for a dry day, but a column that is always zero can also hide a missing feed, so it is listed as a watch item in section 10.

7. SQL Scripts

Run in SQL Server Management Studio against the WeatherData database.

7.1 Create table

```
USE WeatherData;
GO
IF OBJECT_ID('dbo.WeatherHourlyData', 'U') IS NULL
CREATE TABLE dbo.WeatherHourlyData (
    [time]                DATETIME2(0) NOT NULL,   -- local time, Asia/Kolkata
    temperature_2m        FLOAT        NULL,       -- degrees C
    relative_humidity_2m  INT          NULL,       -- percent
    wind_speed_10m        FLOAT        NULL,       -- km/h
    precipitation         FLOAT        NULL,       -- mm
    CONSTRAINT PK_WeatherHourlyData PRIMARY KEY ([time])
);
GO
```

7.2 Reconciliation (rule DQ-09)

```
SELECT COUNT(*) AS total_rows, MIN([time]) AS first_hour, MAX([time]) AS last_hour
FROM dbo.WeatherHourlyData;
```

7.3 Quality checks (rules DQ-01, DQ-02, DQ-03, DQ-06 to DQ-08)

```
-- Nulls (expect 0)
SELECT SUM(CASE WHEN [time]               IS NULL THEN 1 ELSE 0 END) AS null_time,
       SUM(CASE WHEN temperature_2m       IS NULL THEN 1 ELSE 0 END) AS null_temp,
       SUM(CASE WHEN relative_humidity_2m IS NULL THEN 1 ELSE 0 END) AS null_humidity,
       SUM(CASE WHEN wind_speed_10m       IS NULL THEN 1 ELSE 0 END) AS null_wind,
       SUM(CASE WHEN precipitation        IS NULL THEN 1 ELSE 0 END) AS null_precip
FROM dbo.WeatherHourlyData;

-- Duplicate hours (expect no rows)
SELECT [time], COUNT(*) AS copies FROM dbo.WeatherHourlyData
GROUP BY [time] HAVING COUNT(*) > 1;

-- Out-of-range values (expect no rows)
SELECT * FROM dbo.WeatherHourlyData
WHERE relative_humidity_2m NOT BETWEEN 0 AND 100
   OR temperature_2m       NOT BETWEEN -60 AND 60
   OR wind_speed_10m < 0 OR precipitation < 0;

-- Completeness: 24 rows per day
SELECT CAST([time] AS DATE) AS weather_date, COUNT(*) AS hours_loaded,
       CASE WHEN COUNT(*) = 24 THEN 'PASS' ELSE 'CHECK' END AS status
FROM dbo.WeatherHourlyData GROUP BY CAST([time] AS DATE);
```

7.4 Daily summary view

```
GO
CREATE OR ALTER VIEW dbo.vw_WeatherDailySummary AS
SELECT CAST([time] AS DATE)          AS weather_date,
       MIN(temperature_2m)           AS min_temp_c,
       MAX(temperature_2m)           AS max_temp_c,
       ROUND(AVG(temperature_2m), 1) AS avg_temp_c,
       AVG(relative_humidity_2m)     AS avg_humidity_pct,
       MAX(wind_speed_10m)           AS max_wind_kmh,
       SUM(precipitation)            AS total_precip_mm
FROM dbo.WeatherHourlyData
GROUP BY CAST([time] AS DATE);
GO
```

8. Operations Guide

8.1 Prerequisites

- Python with requests, pandas, pyodbc and sqlalchemy installed (pip install requests pandas pyodbc sqlalchemy).
- ODBC Driver 17 for SQL Server installed (confirm with pyodbc.drivers()).
- Network access to api.open-meteo.com and a reachable SQL Server instance with the WeatherData database.

8.2 Run procedure

1. Create the table with script 7.1 (first run only).
2. Open the notebook and run all cells from top to bottom.
3. Confirm the console shows **Pipeline Status: SUCCESS** and the expected record count.
4. Run script 7.2 and 7.3 and confirm the checks return the expected results.

8.3 Monitoring

The log file weather_pipeline.log uses the format `time - level - message`. A healthy run writes these messages in order:

#	Log message
1	Pipeline started
2	Data extracted successfully
3	Data validation completed successfully
4	Data transformation completed
5	CSV and JSON exports completed
6	Weather ETL Pipeline completed successfully
7	Total records processed, and total execution time

8.4 Troubleshooting

Symptom	Likely cause	Action
API extraction failed in log	Network issue or API outage after 3 retries	Check connectivity, re-run later; the error stack trace is in the log
pyodbc cannot connect	Wrong driver or server name, SQL service stopped	Check pyodbc.drivers() and the connection string, start SQL Server
Violation of PRIMARY KEY	Hours already loaded by an earlier run	Skip the load or delete the existing rows for that day
Row count is 48 instead of 24	Notebook run twice with append and no primary key	Remove duplicates, then add the primary key from 7.1

9. Testing and Reconciliation

Results below are taken from the executed notebook.

Test	Description	Expected	Actual	Status
TC-01	API call returns HTTP success	No exception	Success logged	**Pass**
TC-02	Response contains hourly block with 4 variables plus time	5 keys	5 columns built	**Pass**
TC-03	Row count	24	24	**Pass**
TC-04	Null values	0	0	**Pass**
TC-05	Duplicate rows	0	0	**Pass**
TC-06	time column type after conversion	datetime64[ns]	datetime64[ns]	**Pass**
TC-07	JSON and CSV files created	2 files	Export logged	**Pass**
TC-08	Database connection	WeatherData	WeatherData	**Pass**
TC-09	Rows in SQL Server after load	24	24	**Pass**

9.1 Run summary

Source	Open-Meteo API
Location	Hyderabad
Records processed	24
Destination	SQL Server, database WeatherData
Table	dbo.WeatherHourlyData
Status	SUCCESS

10. Known Issues, Risks and Recommendations

10.1 Known issues

ID	Issue	Impact	Recommendation
KI-01	Execution time reported as 3817.12 s: the timer starts in the logging cell and stops in a later cell, so idle time between cells is counted	Misleading run time	Time the stages inside one cell or function with time.perf_counter()
KI-02	Verification preview (step 14) shows 2026-11-07 rows while the API data is for 2026-11-08	Preview is from an earlier run	Re-run the verification cell after a full run
KI-03	Load mode is append with no unique key	A second run duplicates the day	Primary key on time (script 7.1) or load only new hours
KI-04	Precipitation is 0.0 for all rows	Possible missing feed would go unnoticed	Alert when precipitation is zero for several consecutive days

10.2 Risk register

Risk	Likelihood	Impact	Mitigation
API outage or rate limiting	Medium	Medium	Retry policy, raw file kept, re-run on next schedule
Schema change in API response	Low	High	Validation stops the run before load; review column mapping
Duplicate loads	Medium	Medium	Primary key and reconciliation check
Local SQL instance unavailable	Low	Medium	Files are saved before the load, so data is not lost

10.3 Roadmap

- **Automate:** schedule daily with Windows Task Scheduler or Apache Airflow.
- **Scale:** add a location column and loop over several cities; extend the primary key to (location, time).
- **Guard:** make quality rules fail the run automatically and send an alert.
- **Report:** connect Power BI to dbo.vw_WeatherDailySummary.

Appendix A. Glossary

Term	Meaning
ETL	Extract, transform, load: moving data from a source to a target while cleaning it
DataFrame	Pandas table held in memory
PyODBC	Python library that connects to databases through ODBC
SQLAlchemy	Python library that manages database engines and loading
Retry / backoff	Repeating a failed request with a growing wait between attempts
Reconciliation	Comparing rows processed with rows stored in the target
Primary key	Column that uniquely identifies each row in a table